

On the Agile Transformation at Norlys

Beltran Aceves¹

Abstract—Norlys, a Danish cooperative born from the merger of over forty companies, is undergoing an agile transformation. This paper applies Weinberg’s overstructured management theory and Lyytinen, Rose, and Yoo’s hyper-learning framework to analyze how Norlys avoids the managerial thinking that typically derails such initiatives. The DEAL model and the five management fallacies (sequence, choice, modularization, iteration and recursion) identify the patterns Norlys rejects. Distributed gate-keeping, simple design rules, peer networks and external knowledge brokering fill the void. Mintzberg’s organization configurations then expose what remains structurally unresolved. The post-merger diversified form has not stabilized, and the middle line still carries command-and-control reflexes. Behavioral change must be paired with structural realignment of middle management roles.

I. INTRODUCTION

Norlys, a Danish energy and telecommunications company formed from the merger of more than forty organizations, employs roughly 4,500 people serving 3.5 million customers under a cooperative ownership model. An initiative called One 2027 pushes a single agile method across the firm [1]. The transformation takes place against a backdrop of clashing organizational cultures: Telia’s performance-driven approach meeting Norlys’s cooperative tradition [2].

Agile transformations frequently fail. The most common cause lies in the way managers think about the method, not in the method itself. When leaders expect a fixed sequence of steps, treat people as interchangeable parts, or paste the same solution onto every problem, the transformation stalls. Weinberg [3] called this pattern **overstructured management**, an imposition of programmer-style logic onto human systems that refuse to behave like code.

This paper addresses the following problem statement: **How does Norlys’s approach to agile transformation avoid the overstructured management thinking that derails similar initiatives, and what does a structural analysis reveal about the remaining tensions?** The argument runs in two moves. Norlys avoids overstructured management by replacing it with hyper-learning routines. The transformation is structurally incomplete, however, because

the middle line still carries legacy command-and-control thinking.

The analysis proceeds in three steps. First, Weinberg’s DEAL model and five management fallacies identify which overstructured patterns Norlys rejects. Second, Lyytinen, Rose, and Yoo’s [4] hyper-learning framework shows what Norlys adopts in their place. Third, Mintzberg’s [5] organization configurations provide the structural explanation for why the transformation remains uneven. The empirical base is five semi-structured interviews with Norlys stakeholders (an agile coach, a tech lead, a scrum master, a product owner and an enterprise architect) conducted by Aalborg University students in spring 2026 under Patton’s [6] interview guide approach.

II. THEORETICAL FRAMEWORK

A. Overstructured Management

Weinberg [3] proposed two diagnostic tools for managerial thinking. The DEAL model distinguishes successful from unsuccessful managers by where they attribute outcomes. **Successful** managers locate outcomes in **Effort** (what we actually did) and **Ability** (what we are capable of). **Unsuccessful** managers drift toward **Difficulty** (the task was hard) and **Luck** (circumstances betrayed us). The first pair implies control; the second pair implies helplessness.

The five management fallacies describe specific overstructured moves. The **Sequence** fallacy assumes one fixed order of steps, followed faithfully, produces transformation. The **Choice** fallacy flattens complex situations into binary forks. The **Modularization** fallacy treats roles and people as swappable parts. The **Iteration** fallacy assumes that repeating the same action yields progress. The **Recursion** fallacy presses a single frame onto problems that demand several. Together, they describe the programmer’s mindset imported into management, where everything looks deterministic, decomposable and repeatable.

B. Hyper-Learning

Lyytinen, Rose, and Yoo [4] describe how organizations learn in hyper-competitive environments. Four routines operate in parallel. **Distributed gate-keeping** means many people across the firm scan for relevant outside knowledge. **Brokering external knowledge** means hauling that knowledge inside and translating it. **Simple design rules and artifacts** are codified heuristics that point action without dictating

¹Aceves is with Department of Computer Science, Aalborg University, Aalborg, Denmark, baceve21@student.aau.dk.

each step. **Peer networks** enable horizontal knowledge sharing among practitioners.

The headline insight is **parallel ambidexterity**. Exploration (hunting for new knowledge) and exploitation (using what you already know) happen inside the same units, with fast switching between them. Sequential ambidexterity alternates periods. Structural ambidexterity splits them into separate units. Parallel ambidexterity requires trust, simple rules and distributed authority, conditions overstructured management directly smothers.

III. ANALYSIS I: REJECTING OVERSTRUCTURED MANAGEMENT

This first analytical pass identifies which Weinberg fallacies Norlys refuses. Most of the evidence comes from Niels Platz, the agile coach. Seventeen years of practice and a CS background grounded in organizational theory make him an unusually sharp critic of managerial thinking, and his interview reads as a running rebuttal of the patterns Weinberg named.

A. *The DEAL Model: Effort and Ability in Practice*

Niels attributes outcomes to effort and ability. When he describes the transformation, he names what people did: **“We started with behavior: trying to create a common agreement of what we are working on”** [7]. Success rides on a chosen action, not on lucky circumstances. Setbacks, he insists, are capability gaps: **“We cannot just give them free rein, because they do not know what to do with it yet”** [7].

Eva-Maria, the scrum master, applies the same thinking. She calls her role a **“guardrail”**, a check on developer drift rather than a controller. Her attention lands on the team’s ability to self-correct, not on the difficulty of the task. Kasper, the tech lead, attributes technical debt to effort and ability as well. Teams lack time and skills to refactor. The complexity of the code gets less blame than the calendar.

The negative case appears in the data too. The manager who axed every scrum master from the platform teams displays the Difficulty-Luck orientation. He treated the scrum master role as a **difficulty** (a known obstacle to be removed) rather than an **ability** (a capability the teams had not yet built for themselves). Teams lost their knowledge catalysts, and direction collapsed [7]. The manager was eventually removed.

B. *The Sequence Fallacy: Evolution Over Revolution*

Niels rejects the idea of one right sequence for transformation. **“You can take revolutions... or you can take evolutions, which is what we are doing here. Evolutions have less risk because they still allow you to deliver value during the transition”** [7]. SAFe influences appear at Norlys, yet no fixed roadmap was adopted. The organiza-

tion identified stepping stones instead. Q Planning, the TPM role, the feature factory, all openly flagged as imperfect and temporary. **“The TPM role is an antipattern of huge proportions, but it is a stepping stone toward something better”** [7].

The ordering is deliberate. Behavior first, structure second. Niels puts it bluntly: **“You can start by doing an organizational transformation, which always messes up, because you start by just putting people in a new box, and they will act as they did before”** [7]. That is Weinberg’s Sequence Fallacy warning in plain language. Organizational transformation collapses whenever it assumes a deterministic march from structure to behavior.

C. *The Modularization Fallacy: The Scrum Master Case*

The Scrum Master removal case anchors this section. A new manager arrived on the platform teams and fired every scrum master. The reasoning followed a programmer’s instinct: if a function sits outside the current spec, deleting it should be free. Teams built a new tool. The tool missed what customers wanted, and direction evaporated. The organization eventually brought scrum masters back and removed the manager [7].

This is the Modularization Fallacy, plain and simple. The manager pushed a programmer’s logic onto a human system. A function gets deleted if it is not in the spec. But the scrum master role was not a module. It was a catalyst for knowledge sharing, customer connection and team self-organization. The value surfaced only after the role vanished.

D. *The Choice Fallacy: Hybrid Coordination*

Niels refuses the bottom-up versus top-down binary. Asked where the transformation starts, he answered: **“Doing it bottom up will at some point hit a Tayloristic way of thinking... you need to work on all different levels”** [7]. Rasmus, the enterprise architect, agrees: **“It’s a hybrid model. Teams coordinate horizontally during planning and execution, while priorities are set top-down”** [2].

The Choice Fallacy collapses here. The transformation is not fully top-down. It is not fully bottom-up. It runs in parallel, with different mechanisms operating at different levels. The Choice Fallacy demands a single answer. Norlys’s reality requires several.

E. *The Iteration and Recursion Fallacies*

Niels catches himself on the Iteration Fallacy. He admits agile language has curdled into cliché: **“Agile is the baseline for this, and it always will be.”** Repeating the phrase does not move the needle. His remedy is to test the claim rather than amplify it: **“Theory is super important, but let it hit the real world, and let the real world have a veto right”** [7].

The Recursion Fallacy meets a structural counterweight in Niels’s own role. The agile coach crosses boundaries on purpose, between teams, between management levels, between

methodologies. **“Scrum Masters are currently focused on the teams... You also need somebody to work between the lines. That is where the Agile coach works”** [7]. Where management sees one frame, the coach insists on several.

IV. ANALYSIS II: HYPER-LEARNING AS THE REPLACEMENT

The first pass showed what Norlys refuses. The second shows what it adopts instead. Lyytinen, Rose, and Yoo’s [4] four routines appear across the interview data, though the distribution is uneven.

A. Distributed Gate-Keeping

Multiple roles act as gate-keepers at Norlys. The agile coach scans for new practices and filters them in. The scrum master watches team dynamics and flags organizational barriers. The enterprise architect tracks technology trends and turns them into guidelines. Rasmus states it directly: **“Guidelines exist for a reason, security, telemetry, observability. If you choose another approach, you must meet the same standards”** [2]. Gate-keeping is distributed. No single function owns it.

B. Brokering External Knowledge

The post-merger context makes knowledge brokering a primary activity. Norlys absorbed Telia Denmark, which dragged a performance-driven culture into a cooperative frame [2]. Rasmus describes the friction: **“Cultures merge too, and it takes time. For example, when we acquired Telia Denmark, their performance-driven culture mixed with Norlys’ existing culture. Over time, we’ve found a balance, but there were clashes initially.”** The organization translates between knowledge bases continuously.

C. Simple Design Rules

Norlys operates with simple heuristics that steer action without prescribing every move. Story points gauge relative size, not precise forecasts. The **“picture”** heuristic (an 8+ point story is too fuzzy to estimate) pushes Kasper to decompose: **“If something is estimated at eight, it is basically a picture”** [8]. T-shirt sizing and confidence votes coordinate fast, without bureaucratic overhead.

D. Peer Networks

Horizontal knowledge sharing appears in code review practices and IP sprints (Innovation and Planning sprints, the sixth sprint of each quarter dedicated to exploration). Kasper describes code review as bidirectional learning: **“The inexperienced person learns, and the experienced person teaches”** [8]. Eva-Maria frames the IP sprint as dedicated space for exploration alongside delivery [9]. Peer networks operate within and across teams.

E. Parallel Ambidexterity

The signature hyper-learning pattern at Norlys is exploration and exploitation coexisting in the same units. Teams ship in regular sprints (exploitation) and probe in IP sprints (exploration). The switch is fast, and the units are identical. This matches Lyytinen, Rose, and Yoo’s parallel ambidexterity closely, even if Norlys never names it that way. Trust, simple rules and distributed authority are present.

The fit is not perfect. Norlys’s peer networks operate more formally than the organic, fast-twitch model Lyytinen sketches. The Q Planning forum and the TPM role point to a more structured, SAFe-shaped organization than the spontaneous knowledge sharing a pure hyper-learner would tolerate. The transformation is heading toward parallel ambidexterity. It has not arrived.

V. DISCUSSION: A STRUCTURAL INTERPRETATION

The two analyses show that Norlys avoids overstructured management through hyper-learning. The transformation is uneven, however. Some teams operate in mature learning routines. Others remain dependent on the old command-and-control structure. Mintzberg’s organization configurations explain why.

A. The Diversified Configuration

Norlys’s post-merger shape fits Mintzberg’s [5] diversified configuration: a holding company over multiple divisions (energy, telecommunications, streaming), each carrying legacy structures from the merged entities. The coordinating mechanism is standardization of outputs (KPIs, financial targets), and the key part is the middle line (division managers).

Diversified configurations have a known pull. They drift toward **balkanization**, with divisions pulling apart. The One 2027 strategy is a deliberate counter-pull toward integration. The agile transformation is the operational expression of that counter-pull.

B. The Push Toward Adhocracy

The target configuration is what Mintzberg calls the **innovative** form, or adhocracy. Mutual adjustment runs among multidisciplinary teams, with liaison devices (agile coaches, enterprise architects, scrum masters) coordinating across boundaries. The coordinating mechanism shifts from standardization of outputs to mutual adjustment. The key part shifts from the middle line to the support staff and operating core.

Norlys sits in transition. The behavioral changes (rejecting overstructured management, building hyper-learning routines) point toward adhocracy. The structure lags behind.

C. The Middle Line as Resistance Point

Old thinking lingers longest in the middle line. Niels names it directly: **“Middle management is maybe the hardest**

place to change” [7]. The reasons are structural. Middle managers’ authority rested on direct supervision, the coordinating mechanism of the entrepreneurial and machine configurations. Agile swaps direct supervision for mutual adjustment, which erodes the middle manager’s traditional power base.

The Scrum Master case is the clearest illustration. The manager who removed the scrum masters acted on a middle manager’s logic: simplify the structure, strip what is not essential. That logic fits a machine configuration, where standardization of work processes dominates. It is catastrophic in a transition toward adhocracy, where liaison devices carry the coordination load.

D. The Pull of the Technostructure

A second structural force pulls against the transition. The technostructure (finance, compliance and other analyst functions) applies machine-configuration logic to an adhocracy-in-formation. When finance required story points on every closed task for cost calculation, it was standardizing work processes onto a structure that no longer fit [7]. The mandate is rational from a machine perspective (uniform data for cost analysis). It disrupts an adhocracy perspective, where team-level estimation practices need room to breathe.

E. What the Structural Lens Reveals

The hyper-learning analysis shows what Norlys does right. The structural analysis shows what is left undone. The behavioral rejection of overstructured management is real, but it clusters in the operating core (teams, scrum masters, agile coaches) and the strategic apex (IT leadership, the One 2027 program). The middle line and the technostructure still operate with overstructured assumptions.

The unevenness makes sense under this lens. Teams at different maturity levels do not signal failed implementation. They signal incomplete structural transition. Some teams carry the support (agile coaches, peer networks, simple rules) to operate in learning routines. Others stay embedded in structures that pull them back toward command-and-control.

VI. CONCLUSION

Norlys’s agile transformation avoids the overstructured management thinking that derails similar initiatives because its leaders systematically reject Weinberg’s five fallacies. They anchor outcomes in effort and ability. They choose evolution over revolution. They refuse to treat roles as interchangeable modules. They resist the bottom-up versus top-down binary. They accept that iterating the same action does not produce progress.

In place of those fallacies, they build hyper-learning routines. Distributed gate-keeping runs across multiple roles. Brokering translates between merged knowledge bases. Simple design rules (the picture heuristic, confidence votes)

guide action without prescribing it. Peer networks (code review, IP sprints) carry knowledge sideways. Parallel ambidexterity lands in the same units where exploitation already lives.

The structural analysis through Mintzberg exposes what remains. The transition from a diversified post-merger configuration toward an innovative adhocracy is behaviorally advanced but structurally incomplete. The middle line and the technostructure still operate on assumptions imported from the machine configuration. Sustainable transformation requires redefining middle management roles and trimming the technostructure’s reach into team-level practices.

The study leans on interview data from one organization mid-transformation. Transformation advocates (coaches, architects, scrum masters) speak louder here than skeptics. Future work could track how the structural transition unfolds over time, and whether Norlys’s hyper-learning routines hold up as One 2027 matures.

REFERENCES

- [1] Norlys, “Norlys Guest Lecture — Development Methodology and Strategy.” 2026.
- [2] R. Witt, “Interview with Rasmus Witt, Enterprise Architect at Norlys.” 2026.
- [3] G. M. Weinberg, “Overstructured management of software engineering,” *ACM SIGSOFT Software Engineering Notes*, vol. 7, no. 2, pp. 20–28, 1982.
- [4] K. Lyytinen, G. Rose, and Y. Yoo, “Learning in a hyper-competitive environment: A paradoxical case of hyper-learning in the Norlys telecommunications industry,” 2017.
- [5] H. Mintzberg, *Mintzberg on Management: Inside Our Strange World of Organizations*. New York: Free Press, 1989.
- [6] M. Q. Patton, *Qualitative Research and Evaluation Methods*, 3rd ed. Thousand Oaks, CA: Sage Publications, 2002.
- [7] N. Platz, “Interview with Niels Platz, Agile Coach at Norlys.” 2026.
- [8] K. H. Munck, “Interview with Kasper Harry Munck, Lead Developer / Tech Lead at Norlys.” 2026.
- [9] E.-M. Bücking-Rasmussen, “Interview with Eva-Maria Bücking-Rasmussen, Scrum Master at Norlys.” 2026.
- [10] B. F. Jakobsen, “Interview with Birgitte Frendrup Jakobsen, Technical Product Manager at Norlys.” 2026.

VII. APPENDIX: TRACEABILITY FROM THEORETICAL CODES TO EMPIRICAL MATERIAL

This appendix documents how the empirical material—five semi-structured interviews with Norlys stakeholders—was used to map Norlys’s agile software development practice to the three theoretical lenses that structure the main paper: Weinberg’s overstructured management theory, Lyytinen, Rose, and Yoo’s hyper-learning framework, and Mintzberg’s organization configurations. The appendix supports the empirical traceability of the main paper’s argument by linking each theoretical code to interview indicators. Codes with stronger empirical support are documented with multiple indicators; codes with weaker support are documented with one representative indicator; absences are documented as such.

The five interviews are referred to throughout by interviewee role: Agile Coach (Niels Platz) [7], Lead Developer (Kasper Harry Munck) [8], Scrum Master (Eva-Maria Bücking-Rasmussen) [9], Product Owner (Birgitte Frendrup Jakobsen) [10], and Enterprise Architect (Rasmus Witt) [2]. The strength ratings reflect the number of interviews in which the code is evidenced out of five. The Scrum Master removal case recurs across the appendix because it is the single data point that maps onto codes in all three theories and therefore carries the largest share of the main paper’s argumentative weight.

A. Weinberg’s Overstructured Management

a) DEAL — Attribution to Effort or Ability:

Strength: very strong (5/5 interviews).

Indicator 1: Outcomes are attributed to managerial effort and to the team’s growing capability rather than to circumstance. Agile Coach: “**The first part we needed to do was to stop the bleeding. When you talk about Agile, it is nice to write about and read about, and teams are actually the easy ones. Management is the hard part, because they need to do things differently**” [7].

Indicator 2: Decisions are attributed to the developer’s ability to make the call, not to external circumstance. Lead Developer: “**It was a quick conversation with the company about accepting the risk, and then pretty quickly I changed the service that handles that validation and took it out of production. Before, I probably would have started looking into other ways or alternatives**” [8].

Indicator 3: Long career trajectories are narrated in terms of accumulated effort and ability. Enterprise Architect: “**I followed problems upward—from developer to tech lead to solution architect to EA. I kept encountering systemic issues that required broader influence to solve**” [2].

b) DEAL — Attribution to Difficulty or Luck:

Strength: counter-evidence. The pattern is **absent** in the first-person voice of every interviewee.

Indicator 1: The only Difficulty/Luck attribution in the entire data set is **inferred** from a third party. The Agile Coach describes a removed manager who treated the scrum master role as a difficulty to be removed rather than as an ability the teams had not yet built for themselves: “**Then a new manager came in and fired all the Scrum Masters, and things stopped. They created a new application called the [Guba Super Tool], but they did not know what their customers wanted**” [7]. No current Norlys manager in the data attributes failure to task difficulty or to bad luck. The absence is itself a finding: it confirms the main paper’s claim that Norlys avoids the Difficulty/Luck side of the DEAL model.

c) Anti-Sequence — Rejection of a Fixed Transformation Order:

Strength: very strong (5/5 interviews).

Indicator 1: The transformation is framed as evolution rather than revolution. Agile Coach: “**You can take revolutions... or you can take evolutions, which is what we are doing here. Evolutions have less risk because they still allow you to deliver value during the transition. A revolution risks that you die because you did not succeed in your revolution**” [7].

Indicator 2: Practices are knowingly retained as temporary stepping stones even when their architects see them as imperfect. Agile Coach: “**The TPM role is an antipattern of huge proportions, but it is a stepping stone toward something better**” [7]. Product Owner: “**I’m a technical product manager today and it’s not something you can find in any Scrum guide at all. So it’s a role we have implemented here in Norlys... I should be out of job in about three years**” [10].

Indicator 3: The QBR planning rhythm is described as a plan, not as a sequence. Agile Coach: “**Q Planning is, from my point of view, a necessity where we are right now. But it is not something I want in the future. When we get further, it needs to be in a different shape and format. So currently, we are not adhering to one fixed model. We are moving**” [7].

d) Anti-Choice — Refusal of Binary Options:

Strength: very strong (5/5 interviews).

Indicator 1: The bottom-up versus top-down binary is explicitly refused. Agile Coach: “**You need to do all of them. This is a terrible thing, because first you need upper management support. If you do not have that, it will fail. Doing it bottom up will at some point hit a Tayloristic way of thinking... I think you need to work on all different levels**” [7].

Indicator 2: The speed-versus-quality trade-off is treated as a continuum, not a fork. Product Owner: “**I would always say that quality is better than being quick. But it’s also kind of a balance, because you shouldn’t overdo it. So there’s no need to deliver 100% if 95% is enough. So, it’s a balance, I think**” [10]. Scrum Master: “**I would always say that quality is better than being quick. But it’s also kind of a balance**” [9].

Indicator 3: The decision to deviate from process is treated as a spectrum, not a yes/no call. Product Owner: “**How do you assess when to follow the process and when to deviate from it? Oh, it’s a gut feeling. I’m sorry to say, but it’s very much a gut feeling**” [10].

e) Anti-Modularization — Roles Treated as More Than Interchangeable Parts:

Strength: strong (2/5 interviews, but the anchor case is here). The Scrum Master removal is the single most theoret-

ically dense passage in the data and illustrates all three theories simultaneously; it is therefore treated as the central indicator.

Indicator 1. Anchor case — Scrum Master removal. Agile Coach: “Then a new manager came in and fired all the Scrum Masters, and things stopped. They created a new application called the [Guba Super Tool], but they did not know what their customers wanted, what people needed in it, or how to obtain that knowledge... That manager has now been removed, which I am happy about. This is not official wording, but now they are going to have Scrum Masters again. They saw that it simply costs too much to remove the catalyst in the teams” [7].

Indicator 2: The TPM role is treated as a coach, not as a labelled function. Product Owner: “It’s very different teams and it’s very different product owners who come with a very different background... some product owners need help in this area and others need help in this area and other product owners have done it so long that they just kind of want to do it themselves” [10]. The role is not swappable; it adapts to the specific product owner it serves.

f) *Anti-Iteration — Awareness That Repeating the Same Action Is Not Progress:*

Strength: strong (4/5 interviews).

Indicator 1: Ceremonies that stop producing value are removed. Scrum Master: “Something that is part of the scrum handbook would be review and we don’t have that. In my team we have... I think 18 services and applications. So, it’s not really a product team. So, we don’t have one product that we can present to interested parties. And since a lot of it is back-end... how do you present back-end? You can present the code and you can tell it does this but yeah there isn’t really an interest in it” [9].

Indicator 2: The “real world has a veto right” framing rejects the repetition of pre-formed plans. Agile Coach: “Theory is super important, but let it hit the real world, and let the real world have a veto right” [7].

Indicator 3: Past speed-over-quality choices are explicitly not repeated. Product Owner: “when you have to do it with speed instead of the correct way, we have what we call spaghetti-code. So if something goes wrong, it’s very hard to clean up in and it becomes legacy in the speed of time... So fortunately that’s better” [10]. Lead Developer: “You really do not want to be that early in the lifecycle of something where you pick up all the bugs and teething problems of the early versions” [8].

g) *Anti-Recursion — Use of Multiple Frames:*

Strength: medium (3/5 interviews).

Indicator 1: The Agile Coach role is defined as crossing the team/management boundary. Agile Coach: “Scrum

Masters are currently focused on the teams: protecting them and developing them. You also need somebody to work between the lines. That is where the Agile coach works with the different line managers” [7].

Indicator 2: The Scrum Master’s non-IT background is treated as a recursion-fallacy antidote. Scrum Master: “I have to be the one that says: are we sure this is what we want to do? Because sometimes you just get so excited about a new thing” [9].

Indicator 3: The Enterprise Architect’s career arc is itself an anti-recursion trajectory. Enterprise Architect: “I followed problems upward—from developer to tech lead to solution architect to EA. I kept encountering systemic issues that required broader influence to solve” [2].

B. *Lyytinen, Rose and Yoo’s Hyper-Learning*

a) *Distributed Gate-Keeping:*

Strength: very strong (5/5 interviews).

Indicator 1: Multiple roles scan for relevant knowledge. Scrum Master: “My main role is to remove obstacles for the developers and they don’t always say, oh, this is an issue. Help me with this. So sometimes I just have to listen in and just if I hear something where they’re complaining that this shit doesn’t work, then I can jump in and say, how can I help?” [9].

Indicator 2: Gate-keeping is institutionalized in the prioritization funnel. Product Owner: “There is some work happening before the queue planning, which the team is not a part of. So, there’s kind of like a funnel where all the features are put in and then the features that the business thinks are most important, they will go through the funnel and then we get to decide when we can do what within the quarter” [10].

Indicator 3: Architectural gate-keeping is operationalized as friction. Enterprise Architect: “Using our Kubernetes platform is easy and self-service, while going off-platform requires much more effort and justification. Guidelines exist for security, observability, and operations. If you deviate, you must prove it’s cheaper, faster, and equally reliable” [2].

b) *Brokering External Knowledge:*

Strength: strong (4/5 interviews).

Indicator 1: The Telia/Norlys cultural integration is the most explicit instance. Enterprise Architect: “For example, when we acquired Telia Denmark, their performance-driven culture mixed with Norlys’ existing culture. Over time, we’ve found a balance, but there were clashes initially” [2]. Agile Coach: “You have something stale in how they work and how their business model is going to change, and you need to take that into a world owned by customers. If you keep connecting the two

without transforming it, Telia would not make money in the long run” [7].

Indicator 2: Project-level brokering of third-party APIs. Lead Developer: “If we have had the task presented to us a month or so in advance, we have had time to look into it. For instance, an integration to a new third party: we can go through the API specifications, go through the implementation details, and get close to an estimate” [8].

Indicator 3: The TPM role brokers between PO and engineering registers. Product Owner: “Me, for instance, I don’t have any IT education at all. It’s only my experience in Norlys that has gotten me where I am today” [10].

c) *Simple Design Rules:*

Strength: very strong (5/5 interviews).

Indicator 1: The “picture” heuristic for too-large stories. Lead Developer: “We have a rule that if something is estimated at eight, it is basically a picture, meaning it is too uncertain and not ready. It needs to be broken down into smaller pieces” [8].

Indicator 2: The 80/20 capacity rule. Product Owner: “we say, okay, you have 80% of your time to do work from these quarterly meetings where you agree on, and then you have 20% capacity, free capacity to, it can be bugs and it can be new assignments or it can be smaller things that have high value for the business” [10].

Indicator 3: The Q Planning lock and the platform standardization. Agile Coach: “We have a candidate list that is extracted from [unclear]. It is locked 14 days before” [7]. Enterprise Architect: “We currently support both Java and C#. We define a preferred path for new development while still supporting existing expertise” [2].

d) *Peer Networks:*

Strength: strong (4/5 interviews).

Indicator 1: Code review as both quality gate and peer learning. Agile Coach: “Code reviews are both good and bad. They give value in two ways: the inexperienced person learns, and the experienced person teaches” [7].

Indicator 2: Knowledge-sharing sessions as a calendar-blocked routine. Scrum Master: “We do have knowledge sharing sessions. So, we have them in the calendar like once a month or we just say this time is for knowledge sharing but if something pops up like if we worked something more on AI for instance then we would just say okay on Tuesday you will present what you found out” [9]. Lead Developer: “When we build a new service, we make sure everyone knows what is going on with it: walk through the basics, maybe walk through some of the code. If someone did something in a new way, we bring it to our knowledge-sharing meetings” [8].

Indicator 3: The architecture studio as cross-team peer forum. Enterprise Architect: “Proposals are discussed, refined, and approved in our architecture studio with management support” [2].

e) *Parallel Ambidexterity:*

Strength: medium (2/5 interviews). The IP sprint is the central evidence; one interviewee is explicit about deliberately violating the formal structure to enact ambidexterity informally.

Indicator 1: Exploration and exploitation co-exist in the same unit. Agile Coach: “Right now, we have an event that I am in charge of called Q Planning. We have a candidate list that is extracted from [unclear]. It is locked 14 days before. The teams have an innovation and planning sprint. They do their work, then we meet around it, and then we make the plan” [7].

Indicator 2: The Scrum Master breaks the formal structure to enact ambidexterity. Scrum Master: “With the queue planning setup, the quarter is six sprints, and the sixth sprint is the college... what is it IP standing for innovation and planning so in the process that sprint is meant to be the sprint where you innovate and get smarter and stuff but I cheat. And I think it’s more important that if you find something right now that you need to look into, then do that. Why wait for two and a half months?” [9].

C. *Mintzberg’s Organization Configurations*

a) *Middle Line — Managerial Layer:*

Strength: very strong (5/5 interviews). The middle line is the principal site of resistance; it is the empirical anchor of the main paper’s “structural incompleteness” claim.

Indicator 1: Middle management is named as the hardest place to change. Agile Coach: “Middle management is maybe the hardest place to change. Teams can sometimes be delayed in doing it, because they will live in whatever you give them. I think you need to work on all different levels” [7].

Indicator 2: The middle line imposes direct supervision on a context that needs catalysts (same anchor case as Anti-Modularization). Agile Coach: “Then a new manager came in and fired all the Scrum Masters, and things stopped” [7].

Indicator 3: Leadership turnover at the middle line destabilizes programs. Enterprise Architect: “Frequent shifts in leadership caused instability and changing priorities” [2]. Product Owner: “I would move heaven and earth to get that done. And I sometimes make decisions and then I have to say ‘I’m sorry’ afterwards to a leader of any kind” [10].

b) *Operating Core — Direct Work:*

Strength: very strong (5/5 interviews).

Indicator 1: The operating core owns its own quality standard. Agile Coach: **“We create higher standards than what management would demand”** [7].

Indicator 2: Trust in the operating core substitutes for direct supervision. Lead Developer: **“The trust is on the team. There is no micromanagement. It is: come to me if there is a problem. Otherwise you know best what to do”** [8].

Indicator 3: The operating core’s contentment is the Scrum Master’s success metric. Scrum Master: **“I think if it’s a good sprint for me, I think it’s a sprint where the developers are content and have had a good sprint, so if they have good days, it makes me happy. It makes my work easy as well”** [9].

c) *Technostructure — Analyst and Standardization Functions:*

Strength: very strong (5/5 interviews).

Indicator 1: Finance imposes a standardized output on every story. Agile Coach: **“Recently, I had to present something from finance: a new finance model where they asked that every story, before it is closed, should have story points and an application connected to it, so they could know cost”** [7].

Indicator 2: The prioritization funnel is a technostructure artefact. Product Owner: **“in Norlys you take all the demands from all the business areas and then they have a dogfight somewhere in the leadership and then they come out from one to six”** [10].

Indicator 3: Platform teams standardize the engineering environment. Lead Developer: **“We have platform teams that work on improving the developer experience. They maintain a reference framework project that any team can pull down and use, with all the agreed patterns already baked in: pipelines, deployment into our Kubernetes cluster, everything”** [8].

d) *Support Staff — Internal Services:*

Strength: strong (3/5 interviews).

Indicator 1: Agile coaches are a support-staff innovation. Agile Coach: **“You also need somebody to work between the lines. That is where the Agile coach works”** [7].

Indicator 2: The TPM role is a support-staff invention. Product Owner: **“I’m a technical product manager today and it’s not something you can find in any Scrum guide at all. So it’s a role we have implemented here in Norlys to help our product owners”** [10].

Indicator 3: Enterprise architecture is support staff in the formal sense. Enterprise Architect: **“Not as a single role. Platform teams, agile processes, and EAs collectively perform that function”** [2].

e) *Strategic Apex — Top Management:*

Strength: strong (3/5 interviews).

Indicator 1: The IT director enables the transformation. Agile Coach: **“Luckily, we can start from the top. Our IT director, Christian Nord, actually has knowledge of what Agile is”** [7].

Indicator 2: Apex leadership is constrained by the co-operative-ownership structure. Enterprise Architect: **“The representatives can propose and influence decisions. While the CEO and board drive strategy, they must argue their case and get approval from this representative body. Nothing is guaranteed to pass”** [2].

Indicator 3: The apex tries to override the operating core, with mixed success. Product Owner: **“They have tried. One actually succeeded, but fortunately I had some backup, so it was with the same success criteria. Alright, give me the value. So they delivered it. Some hasn’t succeeded. It was just a no”** [10].

f) *Pull to Lead:*

Strength: strong (3/5 interviews).

Indicator 1: Apex enables the agile method. Agile Coach: **“Luckily, we can start from the top. Our IT director, Christian Nord, actually has knowledge of what Agile is”** [7].

Indicator 2: Cooperative ownership is the strongest apex pull in the data. Enterprise Architect: **“It stems from the ownership structure—a cooperative model where around 800,000 households are owners. Representatives elected by these owners have final approval authority on major decisions, which strongly influences company culture”** [2].

g) *Pull to Rationalize:*

Strength: very strong (5/5 interviews).

Indicator 1: Friction is a deliberate design choice. Enterprise Architect: **“Through developer experience and friction. For example, using our Kubernetes platform is easy and self-service, while going off-platform requires much more effort and justification”** [2].

Indicator 2: QBR imposes standardization on planning. Lead Developer: **“With the QBR, it is much clearer: the business has acknowledged that these are the most important things for the next three months, and it gives a much more stable view of the focus areas”** [8].

Indicator 3: Release processes and change gates are imposed from above. Scrum Master: **“In Norlys anyway, there is a change process, so you have to report, and you have to make sure that when we release, we don’t do it in peak hours, for instance”** [9].

h) *Pull to Professionalize:*

Strength: very strong (5/5 interviews).

Indicator 1: The operating core demands justification from the business. Product Owner: **“now it’s not the business who is in a position to say, ‘You have to do this.’”**

Now I can say, ‘I would love to do this, but you have to tell me why’” [10].

Indicator 2: The operating core sets its own quality bar. Agile Coach: “**We create higher standards than what management would demand**” [7].

Indicator 3: Internal experts coordinate the work. Enterprise Architect: “**We have internal experts in billing, commercial products, utilities, etc., and we involve them as needed**” [2].

i) *Pull to Collaborate:*

Strength: very strong (5/5 interviews).

Indicator 1: Q Planning is the central cross-team liaison device. Agile Coach: “**Right now, we have an event that I am in charge of called Q Planning... The teams have an innovation and planning sprint. They do their work, then we meet around it, and then we make the plan**” [7].

Indicator 2: QBR formalizes horizontal coordination. Lead Developer: “**Then we have conversations with the other teams, because each team has their own backlog. We need to agree on timing: if you need this from us in two weeks, we need to start here. If another team needs to deliver to the self-service portal by August, then they need to finish their part here, and we need to deliver ours here. That way we figure out how everything fits together**” [8].

Indicator 3: The architecture studio is a collaboration forum. Enterprise Architect: “**Proposals are discussed, refined, and approved in our architecture studio with management support**” [2].

j) *Pull to Balkanize:*

Strength: counter-evidence. The pattern is **resisted** in most of the data and **evidenced** only once, in the Copenhagen acquisition.

Indicator 1: The post-merger integration actively resists balkanization. Agile Coach: “**We started out by separating them into two bubbles when we prioritized. Then we found out that the connections became more and more frequent between the parts. That meant finding a uniform way for them to relate to each other needed to make sense**” [7].

Indicator 2: One balkanization risk is named. Enterprise Architect: “**Yes. There’s a more competitive culture and higher churn in Copenhagen compared to other regions. Big city dynamics play a role**” [2]. This is the single explicit evidence for Pull to Balkanize in the data.

D. *The Anchor Case (Cross-Cutting)*

Strength: a single narrative instance that maps onto codes in all three theories.

Indicator 1: The Scrum Master removal case illustrates Weinberg’s Anti-Modularization, Lyytinen’s loss of Distributed Gate-Keeping and a Peer-Network node, and

Mintzberg’s middle-line reversal of a Pull to Collaborate. The same passage:

- **Weinberg (Anti-Modularization):** “**Then a new manager came in and fired all the Scrum Masters**” [7].
- **Lyytinen (DGK / PN loss):** “**They created a new application called the [Guba Super Tool], but they did not know what their customers wanted, what people needed in it, or how to obtain that knowledge**” [7].
- **Mintzberg (ML reverses P-COL):** “**That manager has now been removed, which I am happy about. This is not official wording, but now they are going to have Scrum Masters again. They saw that it simply costs too much to remove the catalyst in the teams**” [7].

The case is treated as the anchor of the main paper’s argument because it is the single data point that lets the paper move from parallel application of three theories to sequential application of three theories on the same data point, allowing the logical chain (fallacies → routines → structure) to be walked through in a single narrative.

E. *Counter-Evidence and Absences*

Following the quality criterion of counter-evidence, the following absences are noted explicitly so that the main paper’s claims do not over-extend the data.

- **DEAL-DL is absent in the first-person voice of every interviewee.** No current Norlys manager in the data attributes failure to task difficulty or bad luck. The only trace is the **inferred** attribution of the removed manager in Niels’s account. The main paper’s anti-overstructured claim therefore rests on interviewees’ self-reports—a positive-bias risk that the paper should acknowledge.
- **Parallel Ambidexterity (PA) is the least evidenced Lyytinen code (2/5).** Niels’s IP-sprint description and Eva-Maria’s “I cheat” passage are the only PA evidence. The main paper should not over-claim this routine; the claim should be anchored here, with corroborating rather than universal evidence.
- **Anti-Modularization is the most unevenly evidenced Weinberg code (2/5).** Niels provides the anchor; Birgitte provides a second case (the TPM role). Kasper, Eva-Maria and Rasmus do not address it directly. The paper’s strongest anti-modularization claim is therefore concentrated in two interviewees.
- **Pull to Balkanize is the only Mintzberg code where the paper’s claim is resistance, not presence.** Niels and Birgitte emphasize integration; Rasmus acknowledges the Copenhagen culture’s higher churn. The paper should report both sides.